

```

import os
import logging
import base64
import json
import httpx

from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup, ReplyKey
from telegram.ext import (
    Application, CommandHandler, MessageHandler, CallbackQueryHandler,
    ContextTypes, filters, ConversationHandler
)

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

ANTHROPIC_API_KEY = os.environ.get("ANTHROPIC_API_KEY")
TELEGRAM_TOKEN = os.environ.get("TELEGRAM_TOKEN")

# Conversation states
WAITING_PHOTO, ASK_Q1, ASK_Q2, ASK_Q3, ASK_Q4, SHOW_RESULT = range(6)

# — Brand products catalog — Real Lafeminite products —————
STORE_URL = "https://lafeminitel.com/ar"

LAVMNITE_PRODUCTS = {
    "جافة": [
        {
            "name": "🌿 زيت التين الشوكي - لافمنيت",
            "desc": "ر يُفَتَّح ويُصَفَّى ويشد البشرة الجافة، نتائج ملحوظة من أول استخدام",
            "url": f"{STORE_URL}/منتجات-التين-الشوكي-من-لافيمينت/c1289226895"
        },
        {
            "name": "🧼 صابونة النيل - لافمنيت",
            "desc": "تُنظف وتُرتب وتُشرق البشرة، رغوة كثيفة وريحة تجنن - الأكثر مبيعاً",
            "url": f"{STORE_URL}/منتجات-النيل-من-لافيمينت/c418743493"
        },
    ],
    "دهنية": [
        {
            "name": "🧼 صابونة النيل - لافمنيت",
            "desc": "نظف المسام وتوحد لون البشرة الدهنية، تُقلل الحبوب مع الاستمرار",
            "url": f"{STORE_URL}/منتجات-النيل-من-لافيمينت/c418743493"
        },
        {
            "name": "🧴 منتجات العكر الفاسي - لافمنيت",

```

```

      "desc": "غربي أصيل يسحب الشوائب والزيوت الزائدة، مثالي للبشرة الدهنية",
      "url": f"{STORE_URL}/منتجات-العكر-الفاشي-من-لافيمينت/c424452352"
    },
  ],
  "مختلطة": [
    {
      "name": "مجموعة الحمام المغربي - لافمنيت 🌿",
      "desc": "ون بلدي + كيس تبريم + غسول طبيعي، يُوازن البشرة المختلطة بعمق",
      "url": f"{STORE_URL}/الحمام-المغربي-من-لافيمينت/c127498338"
    },
    {
      "name": "مجموعة وايت بيل - لافمنيت ✨",
      "desc": "ة التفتيح الذكي، تفتيح متوازن دون إفراط - غيّرت قواعد التفتيح",
      "url": f"{STORE_URL}/وايت-بيل/c1616823754"
    },
  ],
  "حساسة": [
    {
      "name": "منتجات العناية بالعين - لافمنيت 🌸",
      "desc": "تركيبة لطيفة خاصة لمنطقة العين الحساسة، تُقلل الهالات والانتفاخ",
      "url": f"{STORE_URL}/منتجات-العناية-بالعين-من-لافيمينت/c235543399"
    },
    {
      "name": "منتجات العناية بالشفاه - لافمنيت 💋",
      "desc": "ترطيب عميق للشفاه الجافة والحساسة، مكونات طبيعية 100% آمنة",
      "url": f"{STORE_URL}/منتجات-العناية-بالشفاه-من-لافيمينت/c416482159"
    },
  ],
  "عادية": [
    {
      "name": "مجموعة وايت بيل - لافمنيت 🌺",
      "desc": "للحفاظ على إشراق بشرتك وتوحيد لونها، تركيبة عضوية مبتكرة",
      "url": f"{STORE_URL}/وايت-بيل/c1616823754"
    },
    {
      "name": "زيت التين الشوكي - لافمنيت 🍷",
      "desc": "يُغذي ويُجدد البشرة العادية، يمنحها إشراقاً وصفاً طبيعياً",
      "url": f"{STORE_URL}/منتجات-التين-الشوكي-من-لافيمينت/c1289226895"
    },
  ],
  # مشكلة البقع والتفتيح
  "بقع": [
    {
      "name": "مجموعة النيلة لتفتيح الجسم - لافمنيت 💚",
      "desc": "صابونة + مقشر + زبدة النيلة، تفتيح ملحوظ من أسبوعين فقط",
      "url": f"{STORE_URL}/منتجات-النيلة-من-لافيمينت/c418743493"
    },
  ],

```

```

{
    "name": "تفتيح المناطق الحساسة والداكنة - لافمنيت 🌟",
    "desc": "مخصص للمناطق الداكنة كالركب والأكواع والإبط، نتائج آمنة وفعّالة"
    "url": f"{STORE_URL}/تفتيح-المناطق-الحساسة-و-الداكنة/c2069951841"
},
],
}

```

```

ROUTINES = {
    "جافة": {
        "بماء الورد", "3" سيروم الهيالورونيك", "4" مرطب كثيف + واقى شمس]: "صباح"
        "3" سيروم الليل بالريتينول الطبيعى", "4" كريم الليل المغذي بزبدة الشيا"]
    },
    "دهنية": {
        "الساليسيليك", "3" سيروم نياسيناميد خفيف", "4" مرطب جل خفيف + واقى شمس"]
        "بدون زيوت x1" دبل كلينزينج (زيت ثم غسول)", "2" مقشر كيميائي 2"] : "مساء"
    },
    "مختلطة": {
        "ورد", "3" سيروم خفيف", "4" مرطب خفيف على الخدين + جل على منطقة"] : "صباح"
        "ملة", "2" غسول لطيف", "3" ترطيب مختلف للمناطق", "4" كريم عيون اختياري"]
    },
    "حساسة": {
        "مهدئ بالألوفيرا", "3" سيروم مهدئ", "4" مرطب هايبوالرجينيك + واقى معدني"]
        "2" غسول فائق اللطافة", "3" سيروم الإصلاح الليلي", "4" كريم بالسيراميد"]
    },
    "عادية": {
        "4" مرطب خفيف + واقى 3", "C" 1" غسول صباحي", "2" تونر فيتامين"] : "صباح"
        "2" تونر مرطب", "3" سيروم الريتينول أو النياسيناميد", "4" مرطب ليلي"]
    },
}

```

— Helpers —————

```

async def analyze_skin_with_claude(image_bytes: bytes) -> dict:
    """Send image to Claude API for skin analysis."""
    b64 = base64.standard_b64encode(image_bytes).decode()
    payload = {
        "model": "claude-sonnet-4-20250514",
        "max_tokens": 1000,
        "messages": [{
            "role": "user",
            "content": [
                {
                    "type": "image",
                    "source": {"type": "base64", "media_type": "image/jpeg", "data": b64}
                }
            ]
        }],
    }

```

```

        "type": "text",
        "text": "" أنت خبير في تحليل البشرة. حلّل هذه الصورة وأجب بـ ""
    }
    {
        "skin_type": "جافة أو دهنية أو مختلطة أو حساسة أو عادية",
        "observations": ["ملاحظة 1", "ملاحظة 2", "ملاحظة 3"],
        "main_concern": "المشكلة الرئيسية",
        "positive": "أبرز نقطة إيجابية في البشرة",
        "confidence": "عالية أو متوسطة أو منخفضة"
    }
}
""منخفضة confidence كن دقيقاً ومشجعاً. إذا لم تكن الصورة واضحة اجعل

```

```

    }
    ]
    ]]
}
async with httpx.AsyncClient(timeout=30) as client:
    resp = await client.post(
        "https://api.anthropic.com/v1/messages",
        headers={"x-api-key": ANTHROPIC_API_KEY, "anthropic-version": "2023-0
        json=payload
    )
    resp.raise_for_status()
    text = resp.json()["content"][0]["text"].strip()
    text = text.replace("`json", "").replace("`", "").strip()
    return json.loads(text)

```

```

def build_skin_keyboard():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("🌵 جافة", callback_data="skin_جافة"),
         InlineKeyboardButton("💧 دهنية", callback_data="skin_دهنية")],
        [InlineKeyboardButton("⚖️ مختلطة", callback_data="skin_مختلطة"),
         InlineKeyboardButton("🌸 حساسة", callback_data="skin_حساسة")],
        [InlineKeyboardButton("🌟 عادية", callback_data="skin_عادية")],
    ])

```

```

def build_concern_keyboard():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("❤️ حب الشباب", callback_data="concern_حب الشباب"),
         InlineKeyboardButton("💧 الجفاف الشديد", callback_data="concern_الجفاف الشديد")],
        [InlineKeyboardButton("👁️ البقع الداكنة", callback_data="concern_البقع الداكنة"),
         InlineKeyboardButton("⚡ التهيج والاحمرار", callback_data="concern_التهيج والاحمرار")],
        [InlineKeyboardButton("🕒 الشيخوخة المبكرة", callback_data="concern_الشيخوخة المبكرة"),
         InlineKeyboardButton("😬 لا توجد مشكلة محددة", callback_data="concern_شيء")],
    ])

```

```
def build_climate_keyboard():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("☀️ اف-الرياح / جو حار جاف", callback_data="climate_اف-الرياح"),
         InlineKeyboardButton("🌧️ حار-رطب / جدة", callback_data="climate_حار-رطب"),
         InlineKeyboardButton("🌴 الطائف / معتدل", callback_data="climate_معتدل"),
         InlineKeyboardButton("🌍 منطقة أخرى", callback_data="climate_أخرى")],
    ])

```

```
def build_budget_keyboard():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("💚 100-200 شهر /د", callback_data="budget_اقتصادي"),
         InlineKeyboardButton("💛 200-400 شهر /د", callback_data="budget_متوسط")],
        [InlineKeyboardButton("💎 400+ شهر /د", callback_data="budget_مميز")],
    ])

```

```
async def build_final_report(context: ContextTypes.DEFAULT_TYPE) -> str:
    d = context.user_data
    skin = d.get("skin_type", "عادية")
    concern = d.get("concern", "لا شيء")
    climate = d.get("climate", "حار-جاف")
    budget = d.get("budget", "متوسط")
    ai_obs = d.get("ai_observations", [])
    ai_positive = d.get("ai_positive", "")
    ai_concern = d.get("ai_main_concern", "")

    routine = ROUTINES.get(skin, ROUTINES["عادية"])
    products = LAVMNITE_PRODUCTS.get(skin, LAVMNITE_PRODUCTS["عادية"])

    climate_tip = {
        "مناخك حار وجاف - ركّزي على الترطيب المكثف وواقي الشمس يومياً حتى في الغيم",
        "⚠️ مناخك حار ورطب - اختاري منتجات خفيفة غير دهنية لتجنب انسداد المسام",
        "✅ تدل: مناخك معتدل نسبياً - الروتين الأساسي سيكفيك مع تعديلات بسيطة صيفاً",
        "📌 أخرى: راعي خصائص مناخك المحلي عند اختيار درجة الترطيب",
    }.get(climate, "")

    obs_text = "\n".join([f"• {o}" for o in ai_obs]) if ai_obs else "• إجاباتك"
    ai_section = ""
    if ai_obs:
        ai_section = f"""


---


📌 ما لاحظته الذكاء الاصطناعي في صورتك: **
{obs_text}
{'✅ ' + ai_positive if ai_positive else ''}
{'⚠️ أبرز مشكلة ' + ai_concern if ai_concern else ''}
"""

```

```

products_text = "\n".join([f"





**نوع بشرتك:** {skin}







**مشكلتك الرئيسية:** {concern}


```

{climate_tip}

{ai_section}



روتين الصباح (4 دقائق):

{morning}



روتين المساء (5 دقائق):

{night}



منتجات لافمנית المناسبة لك:

{products_text}

{extra_products}



نصيحة الأسبوع:

لا تخيري أكثر من منتج واحد في الأسبوع حتى تعرفي ما يناسب بشرتك فعلاً.



تسوقي الآن <https://lafeminitel.com/ar>



استفسار: تواصل معنا عبر المتجر مباشرة

— Handlers —

```

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data.clear()
    keyboard = ReplyKeyboardMarkup(
        [[KeyboardButton("📺 الاضطاعي")],
         [KeyboardButton("❓ (بدون صورة)")]],
    )

```

```

        resize_keyboard=True
    )
    await update.message.reply_text(
        "🌿 *أهلاً بك في بوت لافمنيت للعناية الطبيعية*\n\n"
        "أنا هنا لأحلل بشرتك وأعطيك روتيناً مخصصاً 100% لك! 📄\n\n"
        "اختاري كيف تريدن البدء:",
        parse_mode="Markdown",
        reply_markup=keyboard
    )
    return WAITING_PHOTO

async def handle_photo_choice(update: Update, context: ContextTypes.DEFAULT_TYPE)
    text = update.message.text
    if "تحليل" in text:
        await update.message.reply_text(
            "📷 *أرسلني صورة واضحة لوجهك في ضوء طبيعي*\n\n"
            "💡 نصائح للصورة المثالية:\n"
            "• ضوء طبيعي (قرب النافذة)\n"
            "• بدون مكياج إن أمكن\n"
            "• الوجه كاملاً في الإطار\n\n"
            "سيتم تحليل صورتك بالذكاء الاصطناعي وحذفها فوراً 🗑️",
            parse_mode="Markdown"
        )
        return WAITING_PHOTO
    else:
        await update.message.reply_text(
            "🌿 تمام! سنبدأ بأسئلة سريعة",
            reply_markup=build_skin_keyboard()
        )
        await update.message.reply_text("1️⃣ *ما نوع بشرتك في العادة؟*", parse_mode="Markdown")
        return ASK_Q1

async def handle_photo(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("⌚ *جاري تحليل بشرتك بالذكاء الاصطناعي*\n\n")
    try:
        photo = update.message.photo[-1]
        file = await photo.get_file()
        image_bytes = await file.download_as_bytearray()
        result = await analyze_skin_with_claude(bytes(image_bytes))
        context.user_data["skin_type"] = result.get("skin_type", "عادية")
        context.user_data["ai_observations"] = result.get("observations", [])
        context.user_data["ai_main_concern"] = result.get("main_concern", "")
        context.user_data["ai_positive"] = result.get("positive", "")
        confidence = result.get("confidence", "متوسطة")
        conf_msg = {"منخفضة": "⚠️", "متوسطة": "👍", "عالية": "🌟"}
        conf_msg = {f"الصورة غير واضحة تماماً، سنكمل بالأسئلة": conf_msg["منخفضة"]}
    except Exception as e:
        await update.message.reply_text(f"حدث خطأ: {e}")

```

```

        await update.message.reply_text(
            f"🌀 *نتيجة التحليل الأولي*\n"
            f"نوع بشرتك: **{context.user_data['skin_type']}**\n{conf_msg}\n\n"
            f"📖 الآن سنكمل 3 أسئلة سريعة لنخصص توصياتك أكثر",
            parse_mode="Markdown"
        )
    except Exception as e:
        logger.error(f"Image analysis error: {e}")
        await update.message.reply_text("⚠️ كن من تحليل الصورة، سنكمل بالأسئلة فقط")
        context.user_data["skin_type"] = None

    await update.message.reply_text("1️⃣ *ما نوع بشرتك في العادة؟*\n_ (النتيجة التحليل)")
    return ASK_Q1


async def handle_q1(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    skin = query.data.replace("skin_", "")
    context.user_data["skin_type"] = skin
    await query.edit_message_text(f"✅ *بشرتك: {skin}*", parse_mode="Markdown")
    await query.message.reply_text("2️⃣ *ما أبرز مشكلة تزعجك في بشرتك؟*", parse_mode="Markdown")
    return ASK_Q2


async def handle_q2(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    concern = query.data.replace("concern_", "")
    context.user_data["concern"] = concern
    await query.edit_message_text(f"✅ *المشكلة الرئيسية: {concern}*", parse_mode="Markdown")
    await query.message.reply_text("3️⃣ *يؤثر المناخ كثيراً على البشرة؟*", parse_mode="Markdown")
    return ASK_Q3


async def handle_q3(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    climate = query.data.replace("climate_", "")
    context.user_data["climate"] = climate
    await query.edit_message_text(f"✅ *منطقتك: {climate.replace('_', ' ')}*", parse_mode="Markdown")
    await query.message.reply_text("4️⃣ *ما ميزانيتك الشهرية للعناية؟*", parse_mode="Markdown")
    return ASK_Q4


async def handle_q4(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query

```

```

await query.answer()
budget = query.data.replace("budget_", "")
context.user_data["budget"] = budget
await query.edit_message_text(f"✅ الميزانية: *{budget}*", parse_mode="Markdc
await query.message.reply_text("⌚ *جاري إعداد تقريرك الشخصي*", parse_mode
report = await build_final_report(context)
restart_kb = InlineKeyboardMarkup([[InlineKeyboardButton("🔄 تحليل جديد", cal
await query.message.reply_text(report, parse_mode="Markdown", reply_markup=re
return SHOW_RESULT

async def handle_restart(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    context.user_data.clear()
    await query.message.reply_text("🌿 /start مرحباً من جديد! أرسلني")
    return ConversationHandler.END

async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("🌿 /start من جديد تم الإلغاء. أرسلني")
    return ConversationHandler.END

# — Main —————

def main():
    app = Application.builder().token(TELEGRAM_TOKEN).build()
    conv = ConversationHandler(
        entry_points=[CommandHandler("start", start)],
        states={
            WAITING_PHOTO: [
                MessageHandler(filters.PHOTO, handle_photo),
                MessageHandler(filters.TEXT & ~filters.COMMAND, handle_photo_choi
            ],
            ASK_Q1: [CallbackQueryHandler(handle_q1, pattern="^skin_")],
            ASK_Q2: [CallbackQueryHandler(handle_q2, pattern="^concern_")],
            ASK_Q3: [CallbackQueryHandler(handle_q3, pattern="^climate_")],
            ASK_Q4: [CallbackQueryHandler(handle_q4, pattern="^budget_")],
            SHOW_RESULT: [CallbackQueryHandler(handle_restart, pattern="^restart$
        ],
        fallbacks=[CommandHandler("cancel", cancel)],
        allow_reentry=True,
    )
    app.add_handler(conv)
    app.add_handler(CallbackQueryHandler(handle_restart, pattern="^restart$"))
    logger.info("🌿 لافمنيت بوت يعمل الآن")

```

```
app.run_polling(drop_pending_updates=True)
```

```
if __name__ == "__main__":  
    main()
```